

Rapport J15 — Clés, intégrité référentielle, normalisation et migration RAW → STAGING

Plan d'exécution SQL/Data — 16 semaines

Semaine : 03

Jour : J15

Date d'exécution : 28 septembre 2026 (J15 du parcours commencé le 14 septembre 2026)

Durée prévue : 90 min

Durée réellement effectuée : 135 min

Écart prévu/réel : +45 min

Fin du timer : arrêtée après validation de J15-18

1. Objectifs de la séance

- Comprendre le rôle des clés primaires et étrangères dans un modèle relationnel.
- Distinguer une clé métier (`source_code`) d'une clé technique (`source_id`).
- Créer un référentiel `analytics.source_systems`.
- Vérifier l'unicité des codes sources et l'auto-génération des identifiants.
- Utiliser des JOIN pour relier les événements RAW au référentiel.
- Détecter les événements orphelins avant une migration.
- Expérimenter un backfill de `source_id`, NOT NULL et une contrainte FOREIGN KEY.
- Démontrer le risque de redondance/incohérence lorsque clé métier et clé technique coexistent.
- Revenir à une couche RAW non normalisée et créer une table STAGING normalisée.
- Migrer les 15 événements RAW vers STAGING en conservant leur identité d'origine.

2. État initial validé

Le dépôt Git était propre et synchronisé avec origin/main. La base utilisée est `sql_training` sous PostgreSQL 17.5.

Dataset RAW de référence :

- `total_rows` = 15
- `source_count` = 4
- `event_type_count` = 6
- `min_event_id` = 1
- `max_event_id` = 18
- `total_value` = 1583.40

La table `raw.training_events` possédait initialement `event_id` comme clé primaire.

3. Notions étudiées

PRIMARY KEY

Une clé primaire identifie de manière unique chaque ligne d'une table.

UNIQUE

Une contrainte UNIQUE empêche plusieurs lignes de partager la même valeur pour une clé métier comme `source_code`.

Clé métier et clé technique

`source_code` représente l'identité métier de la source et est connue avant stockage. `source_id` est une clé technique générée automatiquement et adaptée aux relations entre tables.

FOREIGN KEY

Une clé étrangère assure l'intégrité référentielle : une valeur référencée dans la table enfant doit exister dans la table parent.

JOIN

Un JOIN est une opération effectuée lors d'une requête afin de relier les lignes de plusieurs tables selon une condition.

Backfill

Le backfill consiste à renseigner une nouvelle colonne pour les lignes historiques avant de lui appliquer des contraintes plus strictes.

RAW et STAGING

La couche RAW conserve la donnée telle qu'elle a été reçue, notamment source_system. La couche STAGING peut porter une structure normalisée et contrôlée, notamment source_id.

4. Création du référentiel des systèmes sources

Table créée :

```
CREATE TABLE analytics.source_systems (  
  source_id BIGINT GENERATED ALWAYS AS IDENTITY,  
  source_code VARCHAR(100) NOT NULL,  
  source_name VARCHAR(100) NOT NULL,  
  source_category VARCHAR(50) NOT NULL,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  
  CONSTRAINT source_systems_pkey  
    PRIMARY KEY (source_id),  
  
  CONSTRAINT source_systems_source_code_key  
    UNIQUE (source_code)  
);
```

Un test avec test_system a confirmé :

- source_id = 1
- is_active = TRUE
- le second INSERT identique échoue avec une violation de source_systems_source_code_key
- une seule ligne subsiste.

Le test a ensuite été supprimé avant le chargement métier.

5. Chargement du référentiel

Les quatre systèmes sources ont été chargés :

source_id	source_code	source_name	catégorie
3	manual_psql	Manual PSQL	training
4	microgrid_poc	Microgrid POC	energy
5	sensor_gateway	Sensor Gateway	iot
6	tomra_training	TOMRA Training	industrial

Validation :

- systèmes = 4
- codes distincts = 4
- systèmes actifs = 4
- premier source_id = 3
- dernier source_id = 6

Les identifiants 1 et 2 non réutilisés illustrent le comportement normal d'une séquence/IDENTITY PostgreSQL : la suppression ou l'échec d'une opération ne garantit pas la réutilisation des valeurs.

6. Premier JOIN de validation

Le INNER JOIN entre raw.training_events.source_system et analytics.source_systems.source_code a retourné les 15 événements.

Résultats observés :

- nombre de lignes = 15
- premier event_id = 1
- premier source_name = Manual PSQL
- dernier event_id = 18
- dernier source_name = Microgrid POC

7. Contrôle des orphelins

Un LEFT JOIN suivi de WHERE ss.source_id IS NULL a été utilisé pour rechercher des sources RAW sans correspondance dans le référentiel.

Résultat :

```
orphan_events = 0
```

La migration pouvait donc être préparée sans perte de lignes due à une source inconnue.

8. Expérience de migration dans RAW : ajout et backfill de source_id

Une colonne temporaire a été ajoutée :

```
ALTER TABLE raw.training_events  
ADD COLUMN source_id BIGINT;
```

Avant backfill :

- 15 lignes
- source_id à NULL
- appliquer immédiatement NOT NULL aurait provoqué une erreur.

Backfill :

```
UPDATE raw.training_events AS te  
SET source_id = ss.source_id  
FROM analytics.source_systems AS ss  
WHERE te.source_system = ss.source_code;
```

Résultat :

```
UPDATE 15  
manual_psql      -> source_id = 3  
microgrid_poc    -> source_id = 4  
sensor_gateway   -> source_id = 5  
tomra_training   -> source_id = 6
```

Contrôle :

- total_rows = 15
- rows_with_source_id = 15
- null_source_ids = 0
- distinct_source_ids = 4

Après ce backfill, source_id a pu être passé en NOT NULL.

9. FOREIGN KEY et tests d'intégrité

Contrainte ajoutée :

```
ALTER TABLE raw.training_events  
ADD CONSTRAINT training_events_source_id_fkey  
FOREIGN KEY (source_id)  
REFERENCES analytics.source_systems(source_id);
```

Un test transactionnel avec source_id = 999999 a échoué comme attendu :

```
ERROR: insert or update on table "training_events"  
violates foreign key constraint "training_events_source_id_fkey"
```

La transaction a été annulée et le dataset est resté à 15 lignes.

Un second test avec source_id = 4 a réussi temporairement dans une transaction, faisant passer le total à 16 lignes, puis ROLLBACK a restauré les 15 lignes.

10. Mise en évidence du risque de redondance

Une ligne de test a volontairement utilisé :

- source_system = 'sensor_gateway'
- source_id = 4

Or source_id = 4 référence microgrid_poc.

La contrainte FK seule accepte cette ligne parce que 4 existe bien dans la table parent. Le JOIN par source_id montre alors :

```
event_source_system = sensor_gateway
referenced_source_code = microgrid_poc
```

Conclusion : stocker simultanément une clé métier textuelle et une clé technique sans contrainte de cohérence peut créer deux vérités contradictoires. La transaction a été annulée.

11. Décision de modélisation

Décision retenue :

Conserver raw.training_events dans sa forme RAW avec source_system, et créer une table normalisée des événements dans staging.

La colonne expérimentale source_id a donc été retirée de RAW :

```
ALTER TABLE raw.training_events
DROP COLUMN source_id;
```

Le dataset RAW a été revérifié : 15 lignes, données métier intactes.

12. Création de staging.training_events

DDL final :

```
CREATE TABLE staging.training_events (
    event_id BIGINT NOT NULL,
    source_id BIGINT NOT NULL,
    event_type VARCHAR(50) NOT NULL,
    event_value NUMERIC(12,2),
    event_timestamp TIMESTAMPTZ NOT NULL,
    ingested_at TIMESTAMPTZ NOT NULL,

    CONSTRAINT staging_training_events_pkey
        PRIMARY KEY (event_id),

    CONSTRAINT staging_training_events_source_id_fkey
        FOREIGN KEY (source_id)
        REFERENCES analytics.source_systems(source_id)
);
```

event_id n'est pas GENERATED ALWAYS AS IDENTITY dans STAGING : il s'agit de l'identifiant original déjà créé dans RAW. Sa conservation assure la traçabilité de l'événement entre couches.

La table était vide immédiatement après création : 0 rows.

13. J15-18 — Migration RAW → STAGING

Requête exécutée :

```
INSERT INTO staging.training_events (
    event_id,
    source_id,
    event_type,
    event_value,
    event_timestamp,
    ingested_at
)
SELECT
    te.event_id,
    ss.source_id,
    te.event_type,
    te.event_value,
    te.event_timestamp,
    te.ingested_at
FROM raw.training_events AS te
INNER JOIN analytics.source_systems AS ss
    ON te.source_system = ss.source_code
ORDER BY te.event_id ASC;
```

Résultat :

```
INSERT 0 15
```

Validation finale :

```
total_rows          = 15
rows_with_source_id = 15
null_source_ids     = 0
distinct_source_ids = 4
min_event_id        = 1
max_event_id        = 18
```

Le JOIN de contrôle STAGING → référentiel a restitué les 15 événements avec leur source_code, source_name, event_type et event_value.

14. Erreurs, corrections et apprentissages

Double déclaration potentielle de PRIMARY KEY

Lors de la préparation du DDL STAGING, event_id BIGINT NOT NULL PRIMARY KEY et une contrainte nommée PRIMARY KEY(event_id) ne devaient pas être utilisées simultanément. La version finale conserve la contrainte nommée.

DEFAULT de ingested_at

La forme TIMESTAMPTZ NOT NULL CURRENT_TIMESTAMP n'est pas une déclaration de valeur par défaut valide. Pour STAGING, aucun nouveau DEFAULT n'a été ajouté afin de recopier l'horodatage d'ingestion RAW.

Qualification des colonnes dans le JOIN

La requête de migration finale qualifie explicitement les colonnes : te.event_id, ss.source_id, etc. Cela documente leur provenance et évite les ambiguïtés.

source_id et non source_code comme FK

La clé primaire technique de la table parent (source_id) devient la clé étrangère de la table enfant. source_code reste une clé métier unique.

15. Captures d'écran / preuves

Captures importantes à conserver dans documentation/semaine-03/jour-15/captures/ :

- validation environnement / dataset initial
- structure de raw.training_events
- création et structure de analytics.source_systems
- test UNIQUE sur source_code
- chargement des 4 systèmes sources
- premier INNER JOIN
- contrôle des événements orphelins
- J15-09_add-source-id-column-1.png
- J15-09_add-source-id-column-2.png
- backfill de source_id
- passage en NOT NULL
- ajout/test de la FOREIGN KEY
- test valide + ROLLBACK
- test de redondance/incohérence
- suppression de source_id dans RAW
- J15-17_create-staging-training-events.png
- J15-18_raw-to-staging-migration.png

16. Challenge autonome et validation

Le challenge a consisté à raisonner sur une migration progressive :

1. créer un référentiel source ;
2. vérifier l'unicité ;

3. relier les données existantes ;
4. rechercher les orphelins ;
5. effectuer un backfill ;
6. renforcer les contraintes ;
7. tester les violations ;
8. détecter le risque de redondance ;
9. choisir la bonne couche pour la normalisation ;
10. migrer RAW vers STAGING ;
11. valider les KPI de qualité.

Validation : réussie.

17. KPI de progression J15

KPI	Résultat
Lignes RAW préservées	15/15
Systèmes référencés	4/4
Codes sources distincts	4
Événements orphelins avant migration	0
Lignes migrées vers STAGING	15/15
source_id NULL dans STAGING	0
source_id distincts dans STAGING	4
Plage event_id préservée	1 à 18
Test FK invalide bloqué	Oui
Test transactionnel restauré	Oui
Incohérence de redondance démontrée	Oui

18. Git / commit de clôture

Après placement du script, des captures et des trois rapports dans documentation/semaine-03/jour-15/, effectuer :

```
git status --short
git add .\documentation\semaine-03\jour-15\
git status
git diff --cached --stat
git diff --cached --name-status
```

Commit recommandé :

```
git commit -m "docs: complete J15 relational keys normalization and RAW to STAGING migration"
git push
```

Validation finale :

```
git status
git log --oneline --decorate -8
```

La journée ne sera considérée totalement archivée dans le dépôt qu'après cette validation Git.

19. Bilan temps

- Durée prévue : **90 min**
- Durée réelle : **135 min**
- Dépassement : **+45 min**
- Ratio réel/prévu : **150 %**

Le timer est arrêté à la fin de J15-18. Aucun temps Git ultérieur n'est ajouté à la durée réelle de la séance.

20. Conclusion

J15 marque le passage des requêtes SQL d'analyse vers la modélisation relationnelle et la qualité structurelle des données. La séance a permis de manipuler concrètement clés primaires, clés métier, clés techniques, contraintes UNIQUE, NOT NULL, FOREIGN KEY, intégrité référentielle, JOIN, contrôle des orphelins, backfill et migration entre couches.

Le résultat architectural retenu est cohérent avec une chaîne de données professionnelle : RAW préserve la donnée source, le référentiel ANALYTICS porte l'identité normalisée des systèmes et STAGING reçoit les événements enrichis avec une clé technique fiable.

21. Prochaines étapes

J16 partira de cette base normalisée et poursuivra le plan de la semaine 03. La séance J16 devra démarrer avec son propre timer, son objectif précis et une nouvelle documentation quotidienne.

22. Potentiel de réutilisation

Cette journée peut devenir :

- un tutoriel « Clé métier vs clé technique dans PostgreSQL » ;
- une vidéo « Pourquoi une FOREIGN KEY ne suffit pas toujours à empêcher les incohérences » ;
- un atelier « Migrer progressivement une table existante vers un modèle normalisé » ;
- un module de formation sur PRIMARY KEY, UNIQUE, FOREIGN KEY, backfill et transactions ;
- une démonstration portfolio de migration RAW → STAGING avec contrôles de qualité.